

IAC-25-D5.4.5.x96166

A Novel Simulation Platform for Space Systems Cybersecurity R&D

**Simone Urbano^{a*}, Jacques Girard^b, Louis Lolive^c, Vincent Nicomette^d, Pierre Bacquet^e,
Patrick Hebrard^f, Guillaume Auriol^g, Ludovic Cintrat^h, Benjamin Deporteⁱ, Julien Airaud^j,
Matteo Merialdo^k, Clotilde Szywala^l**

^a Starion Group France, 143 Boulevard René Cassin, F-06200 Nice, s.urbano@stariongroup.eu

^b Thales Alenia Space, 26 Av. Jean François Champollion, F-31100 Toulouse, jacques.girard@thalesalieniaspace.com

^c IRT Saint-Exupéry, 3 Rue Tarfaya, F-31400 Toulouse, louis.lolive@irt-saintexupery.com

^d LAAS-CNRS, 7 Av. du Colonel Roche, F-31400 Toulouse, vincent.nicomette@laas.fr

^e Thales Alenia Space, 26 Av. Jean François Champollion, F-31100 Toulouse, pierre.bacquet@thalesalieniaspace.com

^f Starion Group France, 143 Boulevard René Cassin, F-06200 Nice, p.hebrard@stariongroup.eu

^g LAAS-CNRS, 7 Av. du Colonel Roche, F-31400 Toulouse, gauriol@laas.fr

^h Gatewatcher, 5 Rue Bellini, F-92800 Puteaux, ludovic.cintrat@gatewatcher.com

ⁱ IRT Saint-Exupéry, 3 Rue Tarfaya, F-31400 Toulouse, benjamin.deporte@irt-saintexupery.com

^j CNES, 1 Rdpt Pierre Guillaumat, F-31400 Toulouse, julien.airaud@cnes.fr

^k Nexova Cyber, Rue des Etoiles 140, B-6890 Libin, m.merialdo@nexovagroup.eu

^l IRT Saint-Exupéry, 3 Rue Tarfaya, F-31400 Toulouse, clotilde.szywala@irt-saintexupery.com

* Corresponding author

Abstract

As space systems become increasingly important to modern society, ensuring their cybersecurity and resilience is paramount. This study presents a novel technological solution designed to facilitate and to enhance cybersecurity investigations for space systems. We introduce a new simulation platform, developed within CSS (Cybersecurity for Space Systems) project, led by IRT Saint-Exupéry in Toulouse. The platform, built on NASA's NOS3 simulator, can simulate a single spacecraft or constellations thereof, including flight software, mission control systems and space link communications (using CCSDS protocol stack). In this study a swarm of CubeSats with optical payload serves as the reference mission. The platform is composed of multiple software components: a realistic space system simulator, a cyber-range, penetration testing software and dedicated defense software. The cyber-range, based on Nexova CITEF (Cyber Integration, Test and Evaluation Framework), hosts the simulation in a protected environment where dangerous software can be tested safely. This study details the architecture and contributions of the proposed framework, along with examples and demonstrations of its potential, including the development of an onboard intrusion detection system (IDS) and a selection of cyber attacks that targets specific satellites or the constellation. The platform also allows for the integration of physical devices within the simulation, as showcased by investigations using Gatewatcher NDR solutions. The CSS simulation platform serves as a benchmark for cybersecurity research and development on space systems, providing a valuable tool for the community to advance cybersecurity investigations and to enhance the resilience of space-based infrastructures.

1. Introduction

The number of objects orbiting Earth has massively increased in the last decade. The United Nations Office for Outer Space Affairs (UNOOSA) lists over 15000 objects as of June 2025 [1]. Over 11000 have been launched in the last decade and Starlink, operated by SpaceX, currently owns the largest number of satellites, with potential plans for 42000 satellites. The new space economy is growing fast, and space systems infrastructures are becoming critical assets for our society. At the same time, cybersecurity standards, regulations and process are struggling to follow the trend [2]. As the risks and the attack surface of the space sector increase, the need to work on the cybersecurity and

resilience of such systems is becoming increasingly important. Indeed, as emphasized by ENISA analysis of space threat landscape [3] there exist “a direct risk to the availability of essential services and industries critical to our interconnected world”. The 2022 cyberattack on Viasat's KA-SAT network [4], at the onset of the Ukraine conflict, is a clear wake up call. For the purpose of increasing the cyber resilience of the space domain, several tools are necessary. In this paper, we introduce a technological solution designed to facilitate and to enhance cybersecurity investigations for space systems. A simulation platform, built on open-source software (NASA's NOS3 simulator [5]), that can be used for vulnerability research, mitigation means

development and for training of operators or AI/machine learning algorithms. Indeed, the platform can simulate realistic CCSDS traffic for a constellation of spacecraft. The platform is composed by several software components: a realistic space system simulator, a cyber-range, penetration testing software and dedicated defense software. The development of such platform has been possible in the framework of CSS project (Cybersecurity for Space Systems). CSS is led by IRT Saint-Exupéry in Toulouse with the contribution of several partners: Starion Group, Thales Alenia Space, Gatewacher, French Air Force Academy, CNES, LAAS-CNRS and Nexova Group. In Section 2 a short review of existing simulation solutions is introduced. In Section 3 the simulation platform introduced by CSS project is presented and the main contributions of this platform are detailed. In Section 4 the focus is on security development and testing with a focus on attack and defense capabilities. In particular, in Section 5 several relevant attack scenarios are presented, while in Section 6 some specific mitigation strategies are introduced. Finally, in Section 7 the conclusions and perspectives of this study are proposed.

2. Related works

Multiple proprietary and open source solutions exist for the simulation of a space system, the associated flight/ground software and the communication protocols. Some well known frameworks are developed by space agency as ESA, CNES or NASA. For example one can mention ESA GSTVi [6], ESA SIMULUS [7], ESA MO Nanosat Framework [8], ESA SNS3 [9], CNES Basiles [10] and NASA NOS3 [5]. Other simulation platforms are developed by universities or commercial companies as OS3 [11], Hypatia [12] or Ansys STK [13]. For example, in [14], STK is associated to other tools to evaluate the security of a LEO Constellation. In [15], the GSTVi simulator is used to test a secure communications environment for space missions. In [16], NASA NOS3 [5] is chosen to generate representative threat data. Concerning simulation platform used by big companies of the space and defence sector worldwide, very limited information are available. For example in Europe, Thales Alenia Space uses the DSS (Dynamic Spacecraft Simulator) to train operators before launch and throughout the life of the satellite [17]. In the academic sector, custom or ad-hoc simulators are generally preferred (for example in [18]). It is important to observe that for decades space systems development has been characterized by highly customized (and confidential) solutions, including for communication protocols and simulation means. For this reason, one should acknowledge that it is still a hard task to compare the existing solutions in terms of realism and representativity. The recommenda-

tions introduced by the Consultative Committee for Space Data Systems (CCSDS) and the rise of CubeSats and mega constellations programs are slowly changing this situation in favor of standardization and open-source frameworks. In this study we have chosen to develop a custom cybersecurity simulator based on NOS3. This choice is mainly related to the accessibility, reusability and standardization (CCSDS) of this simulation solution.

2.1 NASA NOS3 and CSS platform

The NASA NOS3 [5] simulator is probably the most famous open source simulator that comes in a single easy to deploy virtual machine including real Flight Software (NASA cFS), Ground Software (GSW) and Flight Dynamics. Moreover, NASA NOS3 components (in particular the OpenSatKit simulator [19][20], that is very similar to NOS3) have already been used for cybersecurity testing in well known competitions as HackASat [21] due to its modularity, relative simplicity and code accessibility. Finally, NOS3 communications are based on CCSDS standards. These characteristics make of NOS3 one of the best candidates for developing a satellite constellation simulator for space systems cybersecurity research and development and for integration into a dedicated cyber-range platform (as in [22]). The framework that is being developed for CSS project, is composed of multiple complex software modules:

- 1) A realistic space system simulator (including FSW, GSW and Flight Dynamics). This component is based on NASA NOS3 with several modifications and new components.
- 2) A Cyber-range platform (to manage high risks scenarios including dangerous software as malwares). This component is based on CITEF (Cyber Integration, Test and Evaluation Framework) platform by Nexova [23].
- 3) Penetration testing software (for example Kali Linux OS) equipped with customized exploits developed for the considered space system.
- 4) Detection and defence software. This is based on Gatewacher probes [24] for the ground segment and a customized Intrusion Detection and Prevention System (IDS/IPS) developed by LAAS-CNRS for the space segment.

CSS platform is able to simulate a realistic space attack scenario and the associated defence strategies. The attack/defence can be tested and adapted in real time to iterate on specific threats and risk scenarios. Finally, the goal is to increase the resilience of NOS3 space system

architecture (that is very common in the space sector in the sense of a main software bus with subscribing applications). More details about NOS3 and the CSS developments will be provided in the next section.

3. Space system simulator

NASA NOS3 simulator¹ is composed by multiple complex software components.

- NASA cFS [25]: highly representative FSW module with a proven records of flying missions. Based on a software bus with applications architecture, it is very flexible and modular, thus enabling reusability and reducing development costs. However, it has not been designed to be secure [26][27].
- OpenC3 COSMOS [28]: it is a suite of applications that can be used to control a set of embedded systems. In NOS3 it is deployed as Mission Control System module and it is deployed as a set of docker containers. It is also a very powerful piece of software defined for reusability and not necessary for security [29].
- NASA 42 [30][31]: it is a well established NASA flight dynamics and visualization tool by Eric Stoneking.
- CryptoLib [32]: it is a well known software based solution to simulate the SDLS/SDLS-EP protocol [33]. This encryption layer is fully open-source and it also comes with its share of vulnerabilities [34][35].
- Hardware simulations: hardware behavior is simulated for the communications between the main software bus and sensors, actuators, payloads, etc. on board the satellite. The default version of NOS3 is based on STF-1 mission components [36].

The development process of CSS platform is based on the following assumptions:

- It should be possible to simulate a realistic mission for a satellite with optical payload: pointing the satellite to a specific location, TC routing to request a picture, picture acquisition on-board, telemetry routing with realistic photo, photo decoding and visualization.
- It should be possible to have different operators/users on different VMs (for red/blue teams activities). For this reason, it has been chosen to use 1 VM for each satellite (based on NASA cFS), 1 VM for each Mission Control System (MCS, ground segment based on COSMOS), 1 VM for the flight dynamics and visualization part (NASA 42).

¹We have chosen NOS3 v1.6.2 at the beginning of CSS project.

- It should be possible to use a standard “tcpdump” command to get a capture (.pcap) of the simulator traffic. The physical layer (Radio Frequency or RF link) is not simulated in NOS3. CCSDS TF are passed over UDP to represent the RF link.

The main contributions of this research study with regard to NASA NOS3 (v1.6.2) are related to the following developments:

- NOS3 code adaptations for multi spacecraft configuration and TC/TM Transfer Frame layer management (NOS3 v1.6.2 is based on SPP layer).
- Introduction of new components for multi spacecraft configuration: ISL/RM (Inter Satellite Link/ Routing Machine) and Front End.
- Introduction of a multipurpose input generator.
- Introduction of predefined library of exploits.
- Introduction of predefined missions for a swarm/constellation of CubeSats with optical payloads.
- Introduction of IDS/IPS (Intrusion Detection System/ Intrusion Prevention System) on board the satellite as part of the FSW.
- Introduction of traffic redirection to on-ground probes and the introduction of such probes in a simulated environment (hardware-in-the-loop simulation). In CSS project the on-ground probes are proprietary probes provided by Gatewatcher [24].
- Introduction of additional ADCS mode to target a specific location on Earth with the onboard camera.
- 3D simulation of what the onboard camera can see (Imager component) to generate realistic pictures and realistic CCSDS traffic into the simulator.
- Introduction of mitigation means based on simple software modifications (i.e. introduction of Time To Leave or TTL, ISL watchdog messages, Critical TCs, simple routing table check and reconfiguration, etc.).

3.1 General Architecture

The simulator architecture is provided in Figure 1, where a multi spacecraft configuration is presented. The communications in CCSDS (over UDP) between one Satellite VM and the MCS VM can be summarized by the diagram in Figure 2.

The new components introduced by this study are described hereafter.

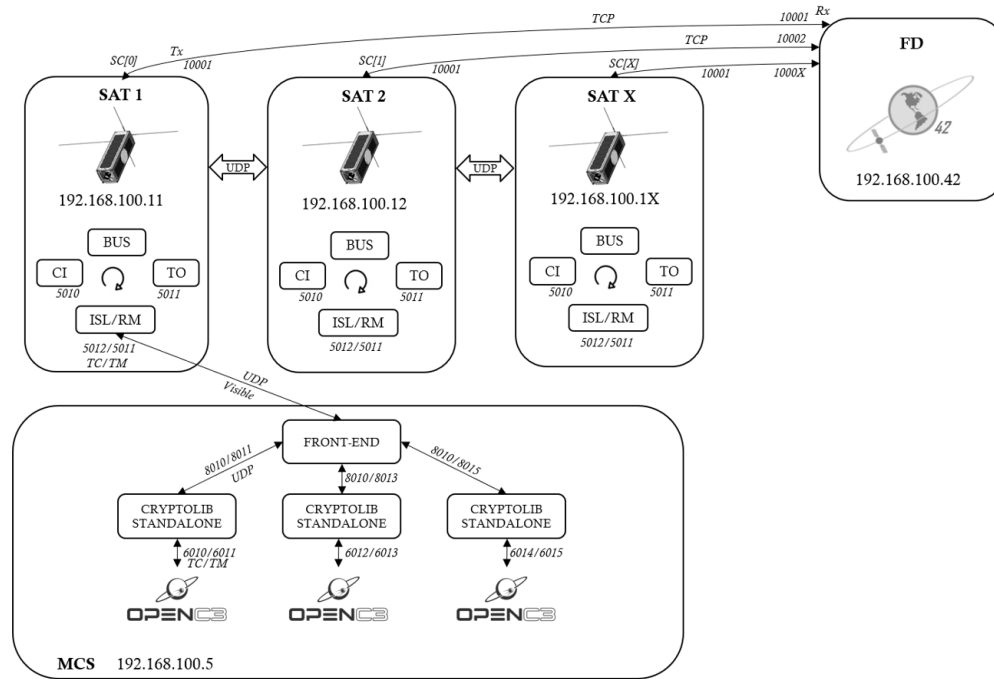


Fig. 1. Overall Architecture of the Simulator.

3.1.1 ISL/RM

ISL/RM implements a Routing Machine and visibility feature. It also simulates simplified radio connections under UDP for ISL and Feeder links. Communication protocol is SPPs in Transfer frames over UDP. Routing and network configurations are static by default. ISL/RM logic is shown in Figure 3.

3.1.2 Front End

Front End (FE) is a new ground component in charge of routing both TM and TC. In addition to ISL/RM component, it implements constellation features: Visibility and Routing. Front End is based on Transfer Frames in both TC and TM. FE receives TC on one port from CryptoLib Standalone; FE receives TM on one port from satellites. Only satellites currently in visibility may send TM to FE. In a future implementation, dynamic routing will be handled. The current version of the simulator considers static routing of the constellation messages. This is fully representative of a real constellation network for a limited amount of time. The routing configuration is defined in simple text files (known by ISL and Front End) and it can be easily modified by the user. ISL routing can also be modified by TC. Dynamic routing and thus variable network configuration depending on the satellite positions and visibility with respect to specific ground stations will be considered in further releases of the simulator.

3.1.3 Imager

The imager is an external simulation tool that enables to visualize in near-real time the camera field of view based on the satellite's attitude. The goal is to obtain realistic photos when a photo capture command is sent to the Arducam (NOS3 optical payload). As input, it receives information about the satellite's position and attitude in ECEF frame, the coordinates of a target and the position of the Sun. As output, it produces an image showing the 3D scene as viewed from the onboard camera. The imager is based on GPU technology, which allows modeling the Earth as a sphere with a texture applied to it, and the 1,000 brightest stars and their respective magnitudes. In Figure 4 an example of the imager output is presented in comparison to NASA 42 output.

3.1.4 Input generator

The user can design scenarios sending multiple commands, or perform fuzzing on TC by using a dedicated tool. A pre-filled JSON file is created with the chosen commands, and the values of the different fields can be customized, allowing for both normal and malicious scenarios to be designed. Different fields of the chosen command can be selected for fuzzing, and the number and frequency of packets sent are user-adjustable.

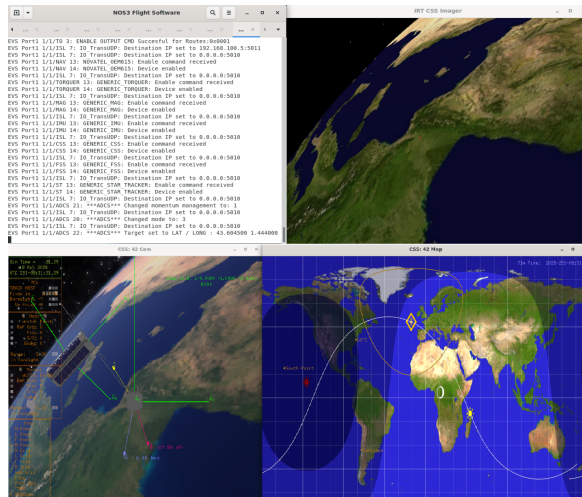


Fig. 4. Example of FSW and Imager process for the Satellite 1 (up), wrt NASA 42 Camera and map (down).

4.1 Risk Analysis

The EBIOS-RM method adopts a digital risk management approach starting from the highest level (major missions of the object under study) to gradually reach the business and technical functions, through the study of possible risk scenarios. It aims to achieve a synthesis between “compliance” and “scenarios”, positioning these two complementary approaches where they provide the greatest added value. A detailed description is available in [38]. The list of the most relevant risk scenarios identified during CSS project are described in Table 3 in Appendix A.

4.2 Threat Model

A large number of research papers focus on ground segment as an attack vector. For example in [42] the vector is a compromised ground station sending illegitimate commands to the satellite. Another strategy for attackers would be using a “rogue” antenna to send commands to the satellite, but accurate tracking of the satellite trajectory would be needed, with a low visibility time frame. In addition, if the satellite and the ground segment use cryptography, sending commands pretending to be the ground station becomes a hard task. Therefore, in general, using the legitimate ground station to access the satellite system seems the best option for an attacker. Another interesting threat consists in compromising the software before being uploaded to the satellite (especially the client payload), for example through some vulnerabilities in the supply chain. This is a common vector in IT security, and as many suppliers are needed to engineer a satellite, the risk increases. Furthermore, as updates for satellite software are performed during operation (especially for the payload), software could be

compromised during operation. Under this threat model, the compromised software can run inside the satellite itself and cannot necessarily be detected by means of a ground IDS analyzing the communications between the satellite and the ground. As such, this leads to the design of an IDS embedded in the satellite itself. The IDS architecture will be described more in detail in Section 6.2, but it is important to mention here that by hypothesis the whole flight system software is trusted and state variables provided by the flight software can be used for detection. However, the payload software embedded in the satellite may be malicious and trigger malicious actions at any moment during the lifetime of the satellite. Overall, in this research work, compromise of the ground segment and compromise of the payload are considered.

5. Attack Samples

The list of available attacks in CSS simulator as of June 2025 is presented in Table 1, the associated risk scenario is also mentioned. The list of available attacks can be largely extended, in particular concerning attacks specific for a constellation of spacecraft. Nevertheless, this can be considered as a good starting point to develop more complex attacks and to start working on specific countermeasures. In the next subsections more details are provided about the attacks implementation.

5.1 Encryption layer

Encryption is very important to contribute to the cybersecurity of a space system, however this protection layer is not immune from vulnerabilities. In particular, as shown in [34] and [35] for the NASA CryptoLib. Indeed, the encryption layer (SDLS-EP):

- It can be bypassed with simple modifications on the TCs packets (see A20).
- It can become an attack vector leading to undesired flight software crash (see A6).
- It can be leveraged to rekey the communications and take full control of a space system (see A22).

The attack A20 is based on the observation that, in the default CryptoLib configuration (up to v1.3.0), 64 Security Associations (SA) and the associated Security Parameter Index (SPI) are initialized, but only a small subset is configured. As a result, several SAs remain initialized but unconfigured, and these “rogue” SAs are valid clear mode SA. Furthermore, there is no check of the security association state that are fixed by default. This results in an improper control of dynamically-managed code resources, that can be used by the attacker to craft a TC that bypasses the encryption layer and it is interpreted in clear mode by

N	Attack Name	Description	Scenario
A1	CI KILL	Send TC to kill CI of SAT X (Sabotage-DoS)	2.3
A2	ISL KILL	Send TC to kill ISL (Sabotage-DoS)	2.3
A3	APPS KILL	Send TC to kill several cFS Apps (Sabotage-DoS)	2.3
A4	FE KILL	Send command to kill Front End on ground (Sabotage-DoS)	5.3
A5	CAM KILL	Send TC to restart CAM App while taking picture (FSW Crash - Sabotage)	2.3
A6	CRYPTO TC CRASH	Send TC to crash CryptoLib on board (DoS)	2.3
A7	TC SB FLOOD	Send TC continuously to flood Software Bus (SB) from ground (DoS)	5.3
A8	APP DELETE	Delete one APP via TC plus the associated .so file in /cf (Sabotage)	2.3
A9	CAM GET	Intercept Camera Payload Data on ground (Confidentiality)	6.3
A10	CAM CORRUPT	Rewrite Camera Payload Data on board (Confidentiality - Sabotage)	5.11
A11	EPS SABOTAGE	Send TCs to discharge the battery via EPS switch all ON plus ADCS sabotage (Integrity)	2.3
A12	ADCS EVIL TC	Send TC to put satellite in rapid rotation around the 3 axis (Sabotage - Integrity)	2.3
A13	EVIL APP FLOOD	Send TC to load malicious App (.so) that flood the SB (DoS)	2.2
A14	CI KILL from APP	Send TC from evil CAM APP to KILL CI (Sabotage-DoS)	2.2
A15	APP DELETE from APP	Send delete all to /cf from CAM app (Sabotage-DoS)	2.2
A16	SCID FE INV	SCID Inversion in Front End (Sabotage)	5.10
A17	SCID FE DOUBLE	SCID Duplication in Front End (Sabotage)	5.10
A18	ISL AUTOLOOP	ISL route tables modification via TC inducing autoloop (DoS)	5.10
A19	ISL LOOP	ISL route tables modification via TC inducing a multi satellites loop (DoS)	5.10
A20	CRYPTO BYPASS	Bypass SDLS by using CryptoLib vulnerabilities (Sabotage - Takeover)	2.3
A21	CRYPTO HIJACK	Load a new encryption key (Takeover)	4.1
A22	FULL CRYPTO HIJACK	Full hijacking procedure based on OTAR PDU commands (Takeover)	4.1

Table 1. List of available exploits (June 2025).

the satellite. An example of malicious TC is provided in Figure 5.

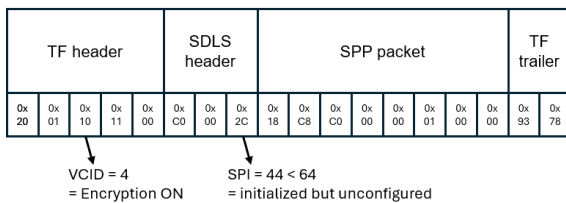


Fig. 5. Example of malicious TC for the attack A20. The SPP packet will be executed on-board as a clear message.

The attacks A21 and A22 leverage also the knowledge of the OTAR PDU procedure [43] and the “states of a security association” to load and to set a new encryption key. In Figure 6 one can see that several TC are needed to modify the SA state and the attacker has to follow a specific procedure, as an operator, to rekey the communications. For example, an attacker can use the procedure described in Algorithm 1.

At the end of the procedure, the operator is denied from accessing the satellite and the attacker is the only one knowing the valid SA and the associated encryption key. This can be considered as some sort of ransomware attack for the space system, because the attacker is in full control

of the system via the encryption layer.

In Figure 7 an example of attack (A21) where a new encryption key is loaded on-board the spacecraft is presented.

NOTE: it is important to observe that in the considered architecture (Figure 1 and 2) the encryption layer is transparent for an attacker sending TC in SPP format instead of a legitimate COSMOS instance.

5.2 Availability

The majority of the considered attacks have an impact on the system availability. Indeed, based on the considered threat model and system architecture, it is very easy for an attacker to compromise the system availability. If the attacker has access to the mission control system network and he wants to induce a service disruption, he/she can simply decide to crash the Front End application on ground that manages the space link and the communications with COSMOS (see A4). If the attacker has the possibility to craft TCs for the constellation, the attack surface becomes even more important. Encryption can reduce the risks but the attacker has still several possibilities as discussed in Section 5.1. The attacker can craft specific TCs to kill critical on-board applications as the Command Ingest (CI) Module and the ISL (see A1 and A2), inducing a network communication disruption. Moreover, it can directly crash the flight software by leveraging existing vulnerabilities in

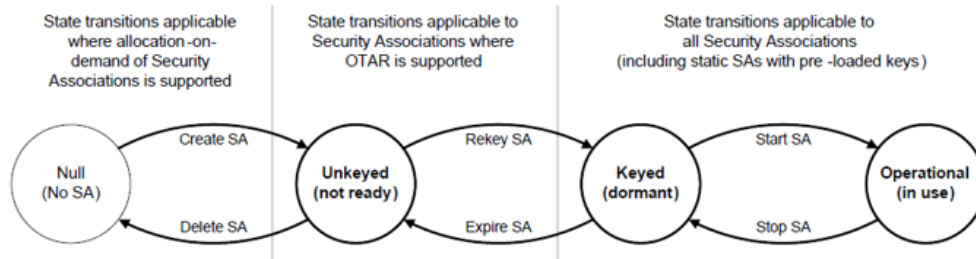


Fig. 6. States of a security association [43].

Algorithm 1: Attack based on OTAR PDU Key Management Procedure

- Step 1 - Load:** Using OTAR PDU command, upload new key (using not-used, all zero Master Key);
- Step 2 - Activate:** Using rogue SA (clear mode), activate key using “Key activation” SDLS-EP as in CCSDS 355.1-B-1 §5.4.2.2;
- Step 3 - Rekey:** Using rogue SA, send following commands for a given target SA;
- “Stop SA” (SDLS-EP §5.5.1.3);
 - “Expire SA” (SDLS-EP §5.5.1.5);
 - “Delete SA” (SDLS-EP §5.5.1.7);
 - “Create SA” (SDLS-EP §5.5.1.6);
 - “Rekey SA” (SDLS-EP §5.5.1.4);
 - “Start SA” to all 7 CryptoLib VCID (SDLS-EP §5.5.1.2);
- Step 4 - Clean:** Using created SA(cyphered mode) ;
- foreach** of remaining 64 SAs except selected one **do**
- “Stop SA”, “Expire SA”, “Delete SA”;
- end**
- Step 5 - Seal:** Eliminate rogue SA used for the attack;
- “Stop SA”, “Expire SA”, “Delete SA”;

the applications and libraries that run on-board the satellites. For example, by using the CVE-2024-44911, one can crash the CryptoLib and as a consequence the whole FSW (see A6). An application kill or restart, as in A3 and A5, can also induce a FSW crash. If the attacker kills a specific application and deletes the associated libraires in the /cf folder on-board, this can result in the obligation for the operator to perform tricky software updates to restore the FSW (reload all the libraries and restart the application). It goes without saying that several types of flooding attacks can be defined, inducing multiple levels of network disruption. If the attacker has the possibility to load new

Fig. 7. FSW output during the attack A21.

software on-board (or the malicious software was loaded on-board before launch) like a malicious CAM application (such as for A13, A14 and A15); the same attacks that can be crafted from ground based on TCs can now be delivered directly from within the satellite on the software bus. This type of attack can be very hard to detect for the operator without the usage of FDIR software or on-board IDS, this aspect will be discussed more in detail in section 6. The attack A1, A2 and A3 are based on the TC command “CFE_ES_STOP_APP” that enable the operator to stop an application on-board. However, no mitigation is in place to reduce the risk of this type of legitimate TC. In section 6 we will see more in details some countermeasures that can be put in place to derisk this type of situation. It is important to observe that for the flooding attacks, the attacker can also consider critical TCs as the “CFE_ES_STOP_APP” or other malicious TCs that induce a crash or a restart of the FSW (such as A6). The implementation of an on-board IDS as described in Section 6.2 is crucial to detect and prevent this type of attacks.

5.3 Confidentiality

In the context of CSS simulator confidentiality is related to the images that are captured, transferred and stored into the MCS during the constellation mission. For an attacker having access to the MCS it is very easy to wait for the picture to be received by telemetry, then decode and copy the image on a different machine. The simulator only represents a simple operator machine that can perform this workflow and A9 represents this simple attack. The attack complexity will evolve as a function of the complexity of the ground station architecture and cybersecurity posture. The picture acquisition simulation on board is based on the following approach: an image file is generated by the Imager component (see Section 3.1.3), the Arducam component decomposes the picture and sends it on ground “line by line” to represent a realistic telemetry traffic. If the attacker directly interferes with the picture telemetry, for example by modifying some bytes, the image will be corrupted and the final .jpg file will be unreadable. For the attacker to modify the picture without the operator noticing, the whole image shall be modified on ground or on-board. The attack A10 replaces the on-board image with a different one to represent this specific case. This is representative of a satellite that has some storage capabilities and that when in visibility of the MCS can transfer the relevant pictures.

5.4 Integrity

For a generic spacecraft with a standard architecture, all the subsystems are critical: electric power, communication, thermal control, attitude and orbit control (ADCS), payload, etc. The loss of one of the subsystems is generally catastrophic for the mission. However, there are some sensors and actuators that have by design some level of redundancy and their loss can be tolerated. For the CubeSat simulated in CSS platform, the subsystem that are more easily exposed to integrity attacks are the electric power system and the attitude and orbit control system. The attack A11 is defined to turn on all power switches on board at the same time to increase energy consumption² and then to set the ADCS to sunsafe mode with a misconfigured target (the ADCS target is set to the opposite of the Sun direction). In general, this type of attack will result in a slow discharge of the battery up to permanent damage (if there is no action from the operator). The attack A12 targets directly the ADCS. Some TCs are sent to switch to passive mode and then to manually set the torques on the 3 axes. The result is a spacecraft in rapid rotation and out of control. All the physical consequences are not simulated in CSS platform, but one can observe that a spacecraft facing rapid uncontrolled rotations may also endure structural and

thermal damage, antenna pointing issues, payload pointing issues, etc. By default NOS3 v1.6.2 only includes reactions wheels and torquers. Thus, there is no direct control on the satellite orbit. However, generic thrusters are available in recent releases of NOS3. The CSS platform user can easily add this type of components and define attacks related to this type of system. For example: send malicious TCs to fire the pyro piercing the propellants tank, TCs to change the satellite orbit, TCs to induce a satellite deorbitation, etc. This type of attacks will be considered in further release of the simulator.

5.5 Constellation

A constellation generally means more redundancy and performance, but it can also mean more complexity and a wider attack surface. To start working on the resilience of constellation networks some simple attacks have been introduced. For example, as the transfer frame header is in clear mode by default, including the spacecraft ID information, the attacks A16 and A17 have been defined. Indeed, it is very easy for the attacker in the MCS to switch or duplicate the SCID information between packets and to sabotage the constellation network without the operator noticing. The final result will depend from the selected commands, but in general the mission will be disrupted. Moreover, if the operator has to modify the satellites attitude and trajectory, this can be catastrophic. A constellation also means the introduction and management of routing tables. A new attack vector that is exploited in A18 and A19. A simple routing table modification can induce loops, mazes and crashes in the constellation network with severe consequences on the mission and even on the space system (see for example [18]). In attack A19, the table 2 is loaded on-board by TC. One can observe that one element of the Table is not coherent: Satellite 2 (Node ID 2) sends message for Satellite 3 to Satellite 1 whereas Satellite 1 sends message for Satellite 3 to Satellite 2. For this reason, when the new malicious routing table is loaded, a simple TC for the Satellite 3 will induce a multi satellite loop. The ISL will probably be squelched by the FSW for all the satellites in the loop, blocking all new communications.

Dest ID		0	1	2	3	4	5	6	7
Node ID	0	0	1	1	1	4	4	4	4
Node ID	1	0	1	2	2	2	2	2	2
Node ID	2	1	1	2	1	4	3	3	3
Node ID	3	2	2	2	3	5	5	5	5
Node ID	4	0	2	2	2	4	5	5	5
Node ID	5	4	4	4	3	4	5	6	7
Node ID	6	5	5	5	5	5	5	6	5
Node ID	7	5	5	5	5	5	5	5	7

Table 2. Example of compromised routing table for a constellation with 7 satellites (0 is ground).

²The switches are not connected to any subsystems by default in NOS3 v1.6.2, a software modification is necessary to increase realism.

6. Mitigations

In this section some mitigation means with respect to the attacks described in Table 1 are discussed. The defence approach is based on the hypothesis that we inherit an existing architecture and we have the possibility afterthought to add:

- Threats related, specific code patch or improvements.
- New components on board that can monitor and influence the traffic.
- External hardware/software on ground that can monitor the system traffic and behavior.

One can argue that security by design or security by component [44] are more robust and effective methods to increase the resilience of space systems. This is true and this type of approach shall be encouraged for all space missions, nevertheless the “threat approach” considered in this paper is very relevant for the industrial practice. Moreover, the threat modelling can be fully reused in a security by design approach. The idea is to propose some simple and effective mitigation means that the user can adapt or reuse for their custom defence strategy. It is important to mention here that some of the mitigation means are still under development, for this reason in the next subsections we will focus more on methodology instead of performance and validation.

6.1 Basic countermeasures

As of June 2025, the following threats related code mitigations are proposed:

- Time To Leave (TTL) information added in front of each transfer frame to avoid multi satellites loops.
- Routing tables verification added in ISL component to avoid multi-satellites loops.
- Critical TC concept is introduced. For example for “CFE_ES_STOP_APP” TC (see Section 5.2), a Message Authentication Code (MAC) is now required for execution.
- Watchdog messages are implemented for space link and ISL verification, enabling dynamic routing table reconfiguration.

It is important to note here that new simple mitigation means and improvements can be proposed in further version of the simulator, for example based on the recommendations provided in [26][27]. One can also observe that as a function of the spacecraft and the mission, spacecraft manufacturers tend to encapsulate the CCSDS protocol to

increase the communications security and reliability (for example by adding a TTL), thus it is important to have also modular and flexible dissectors to analyze the traffic. In Figure 8 an example of FSW output during the attack A19 is presented. The malicious routing table is loaded by the attacker but now the TTL avoid the ISL crash. If the routing table verification is activated it is also possible to prevent erroneous tables to be loaded on board.

```

NOS3 Flight Software
EVS Port1 2/1/DS 35: APP STATE command: state = 1
EVS Port1 2/1/SCH 21: Major Frame Sync too noisy (Slot 1). Disabling synchronization.
EVS Port1 2/1/SC 86: RTS 001 Execution Completed
EVS Port1 2/1/SC 121: Enable RTS group: FirstID=3, LastID=64, Modified=62
EVS Port1 2/1/ISL 7: IO TransUDP: Destination IP set to 0.0.0.0:5010
EVS Port1 2/1/ISL 20: ISL: Configuration command received for file: routeConfig_loop.txt
Route configuration for Sat 2 : 1 1 2 1 4 3 3 3
EVS Port1 2/1/ISL 7: IO TransUDP: Destination IP set to 192.168.100.11:5012
EVS Port1 2/1/ISL 7: IO TransUDP: Destination IP set to 192.168.100.11:5012
EVS Port1 2/1/ISL 7: IO TransUDP: Destination IP set to 192.168.100.11:5012
EVS Port1 2/1/ISL 7: IO TransUDP: Destination IP set to 192.168.100.11:5012
EVS Port1 2/1/ISL 50: ISL: TC packet dropped due to zero ttl
    
```

Fig. 8. FSW output on the Satellite 2 during the attack A19 if the TTL is activated.

6.2 Intrusion Detection and Prevention System

A) Probes

The IDS module embedded in the satellite uses a multi-layer approach with a set of probes (in different locations of the architecture), and a set of detection modules. We investigate the design of three specific probes.

The first probe is located at the arrival of each TC in the satellite (i.e. in the ISL/RM component), and aimed at capturing features of the network traffic exchanged between the satellite and the ground segment: bandwidth, packet inter-arrival time, size of packets, etc. As we assume that cryptography is enabled, this probe is not able to get the content of the packet and thus can only get some characteristics at the network flow level and access the packet header.

The second probe is aimed at reporting information on deciphered TCs coming from the ground, and not sent yet on the Software Bus, to distinguish them from internal messages. This probe is located into the CI component on the NOS3 platform. Both command type and command parameters are captured.

The third probe is aimed at capturing information in the same way as the second one, but is located on the Software Bus. This positioning allows to gather information related to the internal exchanges of the satellite between every component, and thus would enable the detection of a malicious payload. This probe is currently implemented by modifying the software bus itself in order to capture all messages exchanged on the bus along with the source of the message.

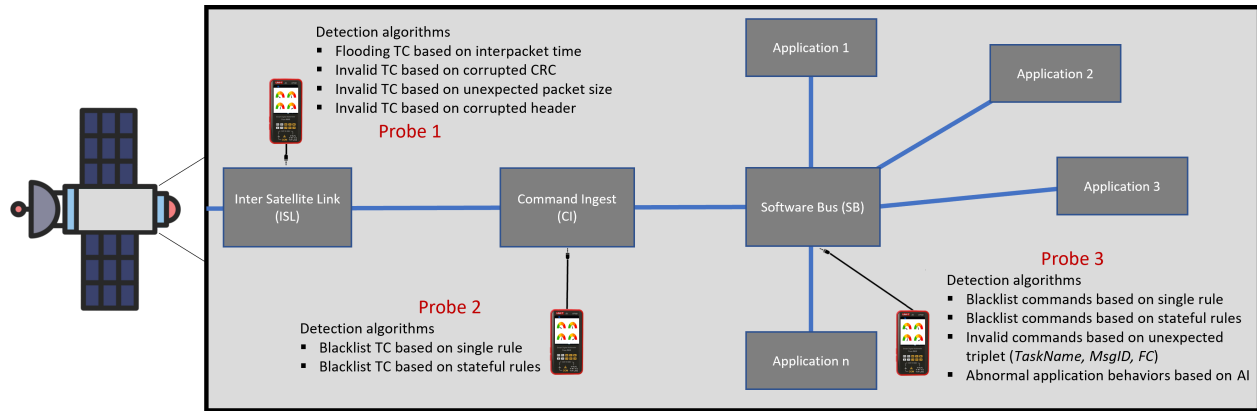


Fig. 9. Embedded probes in NOS3 architecture

Information gathered by these probes is used as an input to the intrusion detection and prevention algorithms. A representation of the location of the probes is given in Fig.9. It is important to note that, to have defense mechanisms as accurate as possible, a standard model of commands sent from the ground which would represent real mission operation is needed. This is the main goal of a prototype mission defined in the MCS VM included in the CITEF scenario.

B) Intrusion detection and prevention strategies

Using the first probe's information, a strategy based on the analysis of the packets characteristics (typically, inter-packet time, size of packets) is considered. The aim is to detect deviations from standard operations behavior, which could indicate an attack. A classic example is a Denial of Service (DoS) attack targeting the satellite, which consists in flooding the satellite with packets coming from the ground, or packets of invalid length which could indicate an attack. As such, this detection mechanism allows to detect the flooding attack such as A7 described in Section 5.2 by comparing the packet inter-arrival time to a threshold, representative of the legitimate communications. We also currently implement an intrusion prevention version with an anti-flooding approach in packets that do not respect the legitimate packet inter-arrival time threshold are dropped (see Figure 10). An other intrusion detection/prevention strategy is currently implemented using the information from this first probe. Even though the content of the packets gathered by the first probe is encrypted, the header is not. Some attacks, such as A6 (based on CVE-2024-44911), can be prevented by monitoring header fields, and in this case detecting packets that include incorrect SPI values. An intrusion detection/prevention strategy using the second and third probe is currently implemented and consists in a

“TC Analyzer”. This component is in charge of checking the validity of the SPP packets, and deploying a black list of dangerous TCs. The aim is to address elementary attacks that use a single malicious TC, both sent from the ground (information gathered by the second probe) or sent on the bus if the payload was compromised (information gathered by the third probe), as well as more complex attacks using a combination of TC. Typically, this TC Analyzer is able to block the attacks that kill applications on board or delete libraries, such as A1/A2 or A15, by means of a filtering rule that can also forbid the use of commands that delete all the configuration files.

The TC Analyzer is also able to detect dangerous combinations of TC. Typically, the attack described in A5 can be dropped by stateful filtering rules that are able to identify the combination of two TC that, taken together, perform a DoS attack towards a specific component of the flight software. These stateful rules are currently being implemented.

Finally, an anomaly detection strategy using the information gathered by the third probe is currently being developed. Our detection strategy consists in establishing a model of the legitimate communications on the software bus and identifying anomalies that could reveal malicious actions over multiple features (packet payload, inter-packet time, header identifiers...). These actions could either be performed by a legitimate software component, on receipt of a malicious TC, or by a malicious satellite payload. The challenge here is to have a model of legitimate messages exchanged on the bus, so that we can detect deviations in these communications. We are currently investigating different AI models, taking into account that our detection algorithms must be embedded in a platform that has limited resources in memory and computation.

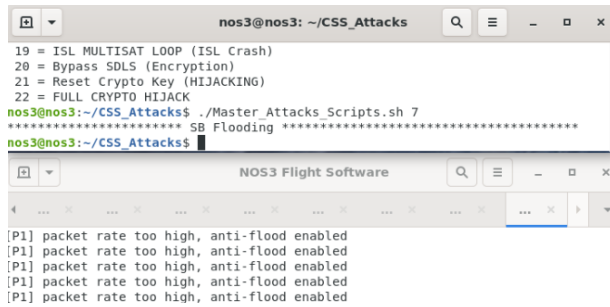


Fig. 10. FSW output on the Satellite 1 during attack 7 if the IDS is activated.

6.3 Ground Probes

Figure 11 depicts a CITEF Scenario including Gatewatcher[24] probes. The probes are physical probes con-

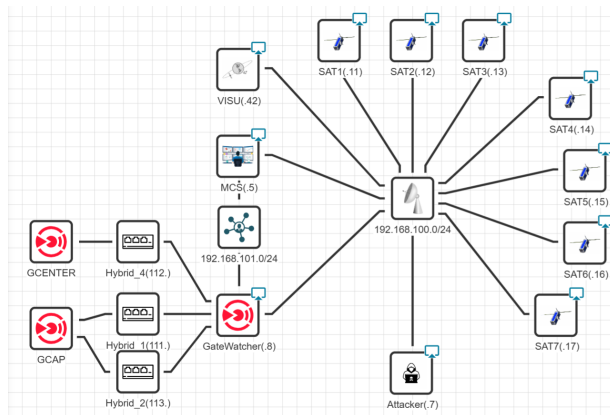


Fig. 11. Example of CITEF Scenario including ground probes.

nected directly to the server that host CITEF and the simulator. The “Hybrid Network” feature of CITEF enables us to feed the emulated traffic directly to the probes and also to receive flags and alarms. Thus, Figure 11 represents a scenario with hardware-in-the-loop (HIL) where we can prototype and test new detection rules for the ground probes. Considering a simple feed from the space link traffic, the Gatewatcher probes can easily detect several of the predefined attack in Table 1. The Gatewatcher probes solution in Figure 11 is composed by 2 elements:

- GCAP: Detection probe at network point; capture, analyze, and send threat data.
- GCENTER: Central platform; manages probes, performs in-depth analysis, provides dashboards and long-term data storage.

These components work together in the Gatewatcher

ecosystem to detect, analyze, and manage threats on the considered space system. In Figure 12 the type of alarms raised in GCENTER console during the attack A22 are showcased. The detection is based on Suricata engine [45].

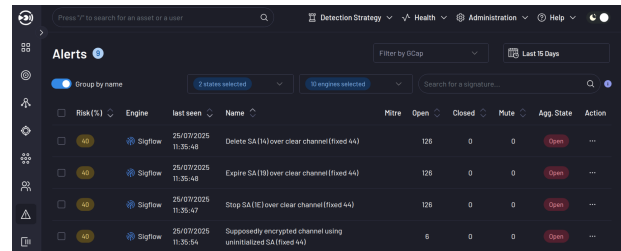


Fig. 12. Example of GCENTER alarms for the attack A22.

6.4 Hybrid Approach

In CSS platform the possibility to integrate on ground and on board information has been investigated to improve the resilience of the space mission. In particular, the IDS can send regular and on demand telemetry to the ground probes in order to enhance the MCS awareness of potential threats. At the same time, the ground probes could inform the onboard IDS of an increased risk level, inducing the activation of more severe rules for the onboard firewall. The interaction can be fully automatic and CSS platform is well adapted for studying and prototyping this type of approach. This hybrid posture between ground and space will be analyzed in greater depth in further studies, nevertheless, it is important to mention here the benefit of this method: 2 heterogeneous and complementary systems, fed with different types of information that can interact to tackle specific threats and to increase the risk awareness of the space mission.

7. Conclusions and perspectives

The main objective of this study is to introduce a benchmark for space systems cybersecurity R&D, along with examples and demonstrations of its potential, including the development of an onboard intrusion detection system (IDS) and a selection of cyber attacks. The idea is to propose a flexible platform for prototyping and testing of attack and defence strategies on a realistic space system. The CSS Simulator is based on NASA NOS3 and it has been adapted to represent a single spacecraft or a constellation. The characteristics, contributions and the potential of the proposed platform for cybersecurity have been described. In particular, the introduction of new components and features that enable the user to develop and to test existing and new scenarios on a representative constellation of spacecraft with optical payloads. Several long-term and short-term mitigations can be proposed to increase the re-

silience of the simulated space mission to cyber attacks (see also [27][46][35]). In this study a set of attacks and mitigations have been proposed that are relevant for the considered architecture and threat model. CSS platform is a starting point for a wide range of research paths in space system cybersecurity. It is possible to work on topic such as automatic attack generation and automatic threat detection/mitigation based on realistic CCSDS traffic. New cybersecurity components, as the intrusion detection and prevention system proposed in Section 6.2, can be developed and validated to increase the system resilience. HIL simulations can be considered in the CITEF environment to prototype detection mechanism on existing hardware. Existing and new algorithms for threat detection can be compared on selected benchmark scenarios. Realistic data sets for surrogate models or for direct training of artificial intelligence algorithms can be generated. Finally, it appears evident that to work on the cyber resilience of today and tomorrow space systems, it is crucial to propose a flexible, realistic and open-source simulation platform.

References

- [1] U.N., *United nations office for outer space affairs, online index of objects launched into outer space*, <https://www.unoosa.org/oosa/osoindex/index.jsp>, 2025.
- [2] G. Falco, "The vacuum of space cyber security," in *2018 AIAA SPACE and Astronautics Forum and Exposition*, 2018, p. 5275.
- [3] European Union Agency for Cybersecurity (ENISA), E. Rekleitis, and M. Adamczyk, *Space threat landscape*, https://www.enisa.europa.eu/sites/default/files/2025-03/Space_Threat_Landscape_Report_fin.pdf, DOI: 10.2824/8841206., 2025.
- [4] N. Boschetti, N. Gordon, and G. Falco, "Space cybersecurity lessons learned from the viasat cyberattack," in *ASCEND 2022*, 4380, 2022.
- [5] NASA, *Nasa operational simulator for small satellites (nos3), v1.6.2*, <https://github.com/nasa/nos3>, 8/2023.
- [6] M. Goetzelmann, N. Lindman, P. Ellsiepen, C. Laroque, M. Niezette, and A. Walsh, "Ground systems test and validation infrastructure-from concept to implementation," in *SpaceOps 2006 Conference*, 2006, p. 5540.
- [7] European Space Agency, *SIMULUS-NG: A New Era for Satellite Simulation*, <https://esoc.esa.int/content/simulus-ng-new-era-satellite-simulation>, 2024.
- [8] European Space Agency, *NanoSat MO Framework*, <https://github.com/esa/nanosat-mo-framework>, 2024.
- [9] SNS3, *Sns3-satellite: Satellite module for ns-3 simulator*, <https://github.com/sns3/sns3-satellite>, SNS-3 was initially developed by Magister Solutions under ESA contract, 2024.
- [10] CNES with SPACEBEL, *Basiles numerical simulation framework*, <https://basiles.fr/>.
- [11] INET Framework, *OS3: Satellite Tracking Library for OMNeT++/INET-Framework*, <https://github.com/inet-framework/os3>, 2015.
- [12] S. Kassing, D. Bhattacharjee, A. B. Águas, J. E. Saethre, and A. Singla, "Exploring the "Internet from space" with Hypatia," in *ACM IMC*, 2020.
- [13] Ansys, *Ansys STK: Digital Mission Engineering Software*, <https://www.ansys.com/products/missions/ansys-stk>, 2024.
- [14] Y. Zhang et al., "Security performance analysis of leo satellite constellation networks under ddos attack," *Sensors*, vol. 22, no. 19, p. 7286, 2022.
- [15] M. Wallum, D. Fischer, J. Nowotnik, Ł. Pieczonka, and M. Tkaczyk, "Sec_lab: A secure communications testbed for space missions," in *Space Operations: Beyond Boundaries to Human Endeavours*, Springer, 2022, pp. 447–469.
- [16] M. G. Sahakian, S. Musuvathy, J. Thorpe, S. Verzi, E. Vugrin, and M. Dykstra, "Threat data generation for space systems," in *2021 IEEE Space Computing Conference (SCC)*, IEEE, 2021, pp. 100–109.
- [17] A. Boudin, F. Brunetti, and P. Connil, "Real-time thermal co-simulation with a simplified fluid loop modelling algorithm," *51st International Conference on Environmental Systems*, 2022.
- [18] Y. Zhang et al., "Energy drain attack in satellite internet constellations," in *2023 IEEE/ACM 31st International Symposium on Quality of Service (IWQoS)*, IEEE, 2023, pp. 1–10.
- [19] OpenSatKit, *OpenSatKit: Core Flight System (cFS) Application Developer's Kit*, <https://opensatkit.github.io/>, 2021.
- [20] D. McComas, "Increasing flight software reuse with opensatkit," in *2018 IEEE Aerospace Conference*, IEEE, 2018, pp. 1–8.
- [21] Hack-A-Sat, *Hack-A-Sat: Capture the Flag Competition*, <https://hackasat.com/>, 2024.

- [22] B. Bailey, “Nasa iv&v’s cyber range for space systems,” in *Annual Ground System Architectures Workshop*, ser. GSAW, 2019.
- [23] Nexova Group, *Cyber Range and Emulation Solutions*, <https://www.nexovagroup.eu/en/cyber-range-and-emulation-solutions>, 2024.
- [24] Gatewatcher, *Gatewatcher: Network Detection and Response (NDR) Solutions*, <https://www.gatewatcher.com/>, 2024.
- [25] NASA, *Core Flight System (cFS)*, <https://github.com/nasa/cFS>, 2024.
- [26] A. Schalk, L. Brodnik, and D. Brown, “Analysis of vulnerabilities in satellite software bus network architecture,” in *2022 IEEE Military Communications Conference*, 2022.
- [27] A. Schalk and D. Brown, “Detection and mitigation of vulnerabilities in space network software bus architectures,” in *2023 IEEE Aerospace Conference*, 2023.
- [28] COSMOS, *Openc3 cosmos*, <https://docs.openc3.com/docs>.
- [29] A. Olchawa, R. Fradique, and M. Starcik, *Openc3 cosmos 6.0.0: A security assessment of an open-source mission framework*, 2024. [Online]. Available: <https://visionspace.com/openc3-cosmos-a-security-assessment-of-an-open-source-mission-framework/>.
- [30] E. Stoneking, *Nasa 42*, <https://github.com/ericstoneking/42>.
- [31] E. Stoneking, “42: An open-source simulation tool for study and design of spacecraft attitude control systems,” NASA, Tech. Rep., 2018.
- [32] NASA, *CryptoLib: CCSDS Space Data Link Security Protocol - Extended Procedures (SDLS-EP)*, <https://github.com/nasa/CryptoLib>, 2025.
- [33] Consultative Committee for Space Data Systems, *Space Data Link Security Protocol*. CCSDS, 2022, CCSDS 355.0-B-2.
- [34] A. Boulaich, A. Olchawa, and M. Starcik, *How to crash a spacecraft – dos through vulnerability in nasa cryptolib v1.3.0*, <https://visionspace.com/crashing-cryptolib/>, 2024.
- [35] A. Boulnois, *Exploring vulnerabilities in the sdds implementation of nasa’s cryptolib*, <https://securitybynature.fr/post/hacking-cryptolib/>, 2024.
- [36] J. Morris et al., “Simulation-to-flight 1 (stf-1): A mission to enable cubesat software-based verification and validation,” in *54th AIAA Aerospace Sciences Meeting*, 2016, p. 1464.
- [37] Gatewatcher, *Gcenter documentation: Motors menu*, https://docs.gatewatcher.com/en/gcenter/2.5.3/102/02_function/motors/motors_menu.html, 2025.
- [38] Agence nationale de la sécurité des systèmes d’information (ANSSI), *La méthode EBIOS Risk Manager*, <https://cyber.gouv.fr/la-methode-ebios-risk-manager>, 2024.
- [39] R. Brobert, *Fuzzing nasa core flight system software*, DEF CON 29 Aerospace Village, Available at: <https://www.youtube.com/watch?v=YtExxB-ayo0>, year = 2021.
- [40] M. Starcik, A. Olchawa, R. Fradique, and A. Boulaich, “Nasa cfs version aquila software vulnerability assessment,” *VisionSpace*, 2025, <https://visionspace.com/nasa-cfs-version-aquila-software-vulnerability-assessment/>.
- [41] M. Starcik, *From the ground up: Space system vulnerabilities*, Presented at CYSAT 2025. [Online]. Available: <https://www.youtube.com/watch?v=P50U2k-Q-qE>.
- [42] J. Theborge, W. Henry, and G. Falco, *Developing scenarios supporting space-based ids*, <https://arc.aiaa.org/doi/10.2514/6.2022-4219>, Oct. 2022. DOI: 10.2514/6.2022-4219.
- [43] Consultative Committee for Space Data Systems, *Space Data Link Security Protocol—Extended Procedures, Recommended Standard, Issue 1*. 2020.
- [44] A. Viswanathan, B. Bailey, K. Tan, and G. Falco, “Secure-by-component: A system-of-systems design paradigm for securing space missions,” in *2024 Security for Space Systems (3S)*, IEEE, 2024, pp. 1–9.
- [45] Open Information Security Foundation (OISF), *Suricata: Network Intrusion Detection and Prevention System*, <https://github.com/OISF/suricata>, 2025.
- [46] A. Prajapati, “Evolution of core flight system (cfs),” in *AIAA SciTech Forum*, 2025.

1. Appendix A: EBIOS RM Risks Scenarios

List of risk scenarios is detailed in Table 3.

Risks Scenarios		
1.1	Intelligence on the satellite system performance	Exfiltration of data via an employee or subcontractor of the prime contractor
1.2		Exfiltration using IT system of the Prime Contractor
1.3		Exfiltration via suppliers of the satellite sytem components
1.4		Exfiltration on the ground system during operations
1.5		Listening ground / space communications
2.1	Destruction / Decommissioning of the Space Segment	Through launch pad means (illegitimate flight configuration)
2.2		Alteration (or update of) of flight software
2.3		Transmission of illegitimate TC from Mission Operation Center
2.4		Using the Ground / Space Link (TTC)
2.5		Perturbation (jamming / spoofing) of GNSS
2.6		Jamming Ground / Space communications (TTC) before and during LEOP
3.1	Reduction in the life of the Space Segment	Alteration of maneuvers through suppliers of satellite system components
3.2		Man in the middle attack on the TTC link
3.3		Injection of illegitimate determination orbit data
4.1	Takeover of the Space Segment	Through Mission Operation Center
4.2		Through compromise of the key management system
4.3		Through compromise of a flight software
4.4		Through Ground/Space TTC link
4.5		Through the use of a degraded mode
4.6		Through inter-satellites links (ISL)
5.1	Disruption / Degradation of the satellite system's mission	Disruption of the network of stations (TTC and PL TM)
5.2		Jamming of Ground / Space Communications (TTC and PL TM)
5.3		Disruption of Ground Communication Network
5.4		Through secondary mission
5.5		Disruption of SatCom mission
5.6		Disruption of the optical or radar imaging mission
5.7		Compromise of time sources
5.8		Sabotage of the PCC through an external OBS service/data supplier
5.9		Jamming / Interrupting of ISL links
5.10		Perturbation of ISL communication routing functions
5.11		Compromise of space segment middleware
6.1	Theft of mission data	Via a security company staff
6.2		Via a supplier (attack on the supply chain)
6.3		On the payload control segment
6.4		On the payload control segment through external user
6.5		On the space segment
6.6		Processed by an OBS external service/product supplier
6.7		Transiting on ISL
7.1	Sabotage of the satellite system in operations	Through a supplier of satellite system components
7.2		Sabotage of MOC through another customer
7.3		Direct sabotage of the satellite system

Table 3. EBIOS RM Risk Scenarios.